

A Lean 4 Formalization of the Bicategorical Yoneda Lemma

Spencer Woolfson

July 8, 2026

Abstract

We present a formalization of the Yoneda lemma for bicategories in Lean 4, using the `Mathlib` library. The classical Yoneda lemma for ordinary categories states a natural bijection $\text{Hom}(y(b), F) \cong F(b)$ between natural transformations out of the representable functor $y(b)$ and elements of $F(b)$. In the bicategorical setting this bijection is upgraded to an equivalence of categories, naturally in both the representing object $b \in \mathcal{B}^{\text{op}}$ and the pseudofunctor $F : \mathcal{B}^{\text{op}} \rightarrow \mathbf{Cat}$. Concretely, we construct pseudofunctors `yonedaPairing` and `yonedaEvaluation` and exhibit them as equivalent via an explicit pair of strong transformations together with unit and counit isomorphisms. A nontrivial aspect of the formalization is universe management: the two sides of the equivalence naturally land in different universe levels, requiring an auxiliary universe-lifting pseudofunctor `catPseudoULift`.

Contents

1	Introduction	3
2	Background	3
2.1	Bicategories	3
2.2	Pseudofunctors	6
2.3	Strong Transformations	6
3	The Bicategorical Yoneda Embedding	6
4	The Main Theorem	7
4.1	The Two Pseudofunctors	7
4.2	The Equivalence	7
5	Universe Considerations	8
6	Design Choices in the Lean Formalization	8
6.1	The BiEquiv Structure	8
6.2	Opacity and Definitional Reductions	8

1 Introduction

The Yoneda lemma is one of the most fundamental results in category theory. In its classical form it states that, for a locally small category $\mathcal{C}$ and a functor $F : \mathcal{C}^{\text{op}} \rightarrow \mathbf{Set}$, there is a natural bijection

$$\text{Hom}_{\mathbf{Set}^{\mathcal{C}^{\text{op}}}}(y(c), F) \cong F(c)$$

where $y : \mathcal{C} \rightarrow \mathbf{Set}^{\mathcal{C}^{\text{op}}}$ is the Yoneda embedding. The lemma has wide-ranging consequences: it identifies objects with the functors they represent, it underlies the theory of adjunctions, and it is the engine behind most universal constructions in algebra and geometry.

In the bicategorical setting, the appropriate notion of “functor” is a *pseudofunctor* (or homomorphism of bicategories), natural transformations are replaced by *strong (pseudo-natural) transformations*, and the Yoneda bijection is promoted to an *equivalence of categories*: for a bicategory $\mathcal{B}$ with Yoneda embedding $\mathcal{Y} : \mathcal{B} \rightarrow [\mathcal{B}^{\text{op}}, \mathbf{Cat}]_{\text{ps}}$, we have

$$\mathbf{StrongTrans}(\mathcal{Y}_0(b), F) \simeq F(b) \tag{1}$$

naturally in $b \in \mathcal{B}^{\text{op}}$ and $F : \mathcal{B}^{\text{op}} \rightarrow \mathbf{Cat}$.

This paper reports on a machine-checked proof of (1) carried out in Lean 4 using Mathlib [The Mathlib Community, 2024]. The formalization closely follows the internal categorical description found in Johnson and Yau [2021] and Leinster [2004]: rather than constructing the equivalence object-by-object, we exhibit it as a biequivalence `yonedaPairing` $\simeq$ `yonedaEvaluation` of pseudofunctors $\mathcal{B}^{\text{op}} \times [\mathcal{B}^{\text{op}}, \mathbf{Cat}] \rightarrow \mathbf{Cat}$ (Section 4).

Organization. Section 2 recalls the relevant background on bicategories, pseudofunctors, and strong transformations, fixing notation. Section 3 reviews the bicategorical Yoneda embedding as it appears in Mathlib. Section 4 states and proves the main theorem. Section 5 discusses the universe issues that arise in the formalization and the auxiliary construction `catPseudoULift`. Section 6 highlights design choices in the Lean code. Section 7 discusses directions for future formalization work.

2 Background

2.1 Bicategories

We work with *bicategories* in the sense of Bénabou [Bénabou, 1967].

Definition 2.1. A *bicategory* $\mathcal{B}$ consists of:

- (i) a collection of *objects* $\text{obj}(\mathcal{B})$;
- (ii) for each pair of objects $a, b \in \mathcal{B}$, a category $\mathcal{B}(a, b)$ whose objects are *1-morphisms* and whose morphisms are *2-morphisms*;
- (iii) *horizontal composition* functors $\circ : \mathcal{B}(b, c) \times \mathcal{B}(a, b) \rightarrow \mathcal{B}(a, c)$;
- (iv) *identity* 1-morphisms $\text{id}_a \in \mathcal{B}(a, a)$;

(v) natural isomorphisms

$$\alpha_{f,g,h} : (h \circ g) \circ f \cong h \circ (g \circ f)$$

$$\lambda_f : \text{id} \circ f \cong f$$

$$\rho_f : f \circ \text{id} \cong f$$

called the *associator* and the *unitors*, satisfying the pentagon and triangle axioms.

In Lean/Mathlib the bicategory axioms are bundled in the `Bicategory` typeclass. We use the notation $f \triangleright \sigma$ (`whiskerLeft`) and $\sigma \triangleleft f$ (`whiskerRight`) for horizontal composition of a 1-morphism with a 2-morphism.

```

1 class Bicategory (B : Type u) extends CategoryStruct.{v} B where
2   /- The category structure on the collection of 1-morphisms -/
3   homCategory :      a b : B, Category.{w} (a → b) := by
4     infer_instance
5   /- Left whiskering for morphisms -/
6   whiskerLeft {a b c : B} (f : a → b) {g h : b → c} (η : g → h) : f >>
7     g → f >> h
8   /- Right whiskering for morphisms -/
9   whiskerRight {a b c : B} {f g : a → b} (η : f → g) (h : b → c) : f
10     >> h → g >> h
11   /- The associator isomorphism: '(f >> g) >> h ≅ f >> g >> h' -/
12   associator {a b c d : B} (f : a → b) (g : b → c) (h : c → d) : (f >>
13     g) >> h ≅ f >> g >> h
14   /- The left unitor: '1 a >> f ≅ f' -/
15   leftUnitor {a b : B} (f : a → b) : 1 a >> f ≅ f
16   /- The right unitor: 'f >> 1 b ≅ f' -/
17   rightUnitor {a b : B} (f : a → b) : f >> 1 b ≅ f
18   -- axioms for left whiskering:
19   whiskerLeft_id :      {a b c} (f : a → b) (g : b → c), whiskerLeft f
20     (1 g) = 1 (f >> g) := by
21     cat_disch
22   whiskerLeft_comp :
23     {a b c} (f : a → b) {g h i : b → c} (η : g → h) (θ : h → i),
24     whiskerLeft f (η >> θ) = whiskerLeft f η >> whiskerLeft f θ := by
25     cat_disch
26   id_whiskerLeft :
27     {a b} {f g : a → b} (η : f → g),
28     whiskerLeft (1 a) η = (leftUnitor f).hom >> η >> (leftUnitor g).
29     inv := by
30     cat_disch
31   comp_whiskerLeft :
32     {a b c d} (f : a → b) (g : b → c) {h h' : c → d} (η : h → h')
33     ,
34     whiskerLeft (f >> g) η =
35     (associator f g h).hom >> whiskerLeft f (whiskerLeft g η) >> (
36       associator f g h').inv := by
37     cat_disch

```

```

30  -- axioms for right whiskering:
31  id_whiskerRight :      {a b c} (f : a→b) (g : b→c), whiskerRight
    (1 f) g = 1 (f >> g) := by
32    cat_disch
33  comp_whiskerRight :
34    {a b c} {f g h : a→b} (η : f→g) (θ : g→h) (i : b→c),
35    whiskerRight (η >> θ) i = whiskerRight η i >> whiskerRight θ i :=
    by
36    cat_disch
37  whiskerRight_id :
38    {a b} {f g : a→b} (η : f→g),
39    whiskerRight η (1 b) = (rightUnitor f).hom >> η >> (rightUnitor g
    ).inv := by
40    cat_disch
41  whiskerRight_comp :
42    {a b c d} {f f' : a→b} (η : f→f') (g : b→c) (h : c→d)
    ,
43    whiskerRight η (g >> h) =
44    (associator f g h).inv >> whiskerRight (whiskerRight η g) h >>
    (associator f' g h).hom := by
45    cat_disch
46  -- associativity of whiskerings:
47  whisker_assoc :
48    {a b c d} (f : a→b) {g g' : b→c} (η : g→g') (h : c→d)
    ,
49    whiskerRight (whiskerLeft f η) h =
50    (associator f g h).hom >> whiskerLeft f (whiskerRight η h) >> (
    associator f g' h).inv := by
51    cat_disch
52  -- exchange law of left and right whiskerings:
53  whisker_exchange :
54    {a b c} {f g : a→b} {h i : b→c} (η : f→g) (θ : h→i),
55    whiskerLeft f θ >> whiskerRight η i = whiskerRight η h >>
    whiskerLeft g θ := by
56    cat_disch
57  -- pentagon identity:
58  pentagon :
59    {a b c d e} (f : a→b) (g : b→c) (h : c→d) (i : d→e),
60    whiskerRight (associator f g h).hom i >>
61    (associator f (g >> h) i).hom >> whiskerLeft f (associator g
    h i).hom =
62    (associator (f >> g) h i).hom >> (associator f g (h >> i)).hom
    := by
63    cat_disch
64  -- triangle identity:
65  triangle :
66    {a b c} (f : a→b) (g : b→c),

```

```

67      (associator f (1 b) g).hom >> whiskerLeft f (leftUnitor g).hom
68      = whiskerRight (rightUnitor f).hom g := by
69      cat_disch

```

2.2 Pseudofunctors

Definition 2.2. A *pseudofunctor* $F : \mathcal{B} \rightarrow \mathcal{C}$ between bicategories consists of:

- (i) an object map $F_0 : \text{obj}(\mathcal{B}) \rightarrow \text{obj}(\mathcal{C})$;
- (ii) for each $a, b \in \mathcal{B}$, a functor $F_{a,b} : \mathcal{B}(a, b) \rightarrow \mathcal{C}(Fa, Fb)$;
- (iii) coherence isomorphisms $F(\text{id}_a) \cong \text{id}_{Fa}$ and $F(g \circ f) \cong F(g) \circ F(f)$, subject to coherence axioms.

In Mathlib pseudofunctors are called `Pseudofunctor` and are written $\mathbf{B} \rightarrow^{\mathbf{P}} \mathbf{C}$ in Lean.

2.3 Strong Transformations

Definition 2.3. Let $F, G : \mathcal{B} \rightarrow \mathcal{C}$ be pseudofunctors. A *strong (pseudo-natural) transformation* $\eta : F \rightarrow G$ consists of:

- (i) for each object $a \in \mathcal{B}$, a 1-morphism $\eta_a : Fa \rightarrow Ga$;
- (ii) for each 1-morphism $f : a \rightarrow b$ in $\mathcal{B}$, an invertible 2-morphism (the *naturality isomorphism*) $\eta_f : G(f) \circ \eta_a \cong \eta_b \circ F(f)$;
- (iii) coherence conditions with respect to identities and composition.

The category whose objects are strong transformations $F \rightarrow G$ and whose morphisms are *modifications* is denoted $\mathbf{StrongTrans}(F, G)$ and written `Pseudofunctor.StrongTrans F G` in Lean.

3 The Bicategorical Yoneda Embedding

Fix a bicategory $\mathcal{B}$. The *Yoneda embedding* for bicategories is a pseudofunctor

$$\mathcal{Y} : \mathcal{B} \longrightarrow [\mathcal{B}^{\text{op}}, \mathbf{Cat}]_{\text{ps}}$$

where $[\mathcal{B}^{\text{op}}, \mathbf{Cat}]_{\text{ps}}$ denotes the bicategory of pseudofunctors $\mathcal{B}^{\text{op}} \rightarrow \mathbf{Cat}$. This is provided by Mathlib as `Bicategory.yoneda` (and the component at an object b is written `yoneda_0 b`).

Concretely, $\mathcal{Y}_0(b) : \mathcal{B}^{\text{op}} \rightarrow \mathbf{Cat}$ is the pseudofunctor that

- sends an object $a \in \mathcal{B}^{\text{op}}$ to the hom-category $\mathcal{B}(a, b)$;
- sends a 1-morphism $f : a' \rightarrow a$ in $\mathcal{B}^{\text{op}}$ (i.e. $f : a \rightarrow a'$ in $\mathcal{B}$) to the precomposition functor $f^* : \mathcal{B}(a, b) \rightarrow \mathcal{B}(a', b)$, $g \mapsto g \circ f$.

4 The Main Theorem

4.1 The Two Pseudofunctors

The *pairing pseudofunctor*

$$P : \mathcal{B}^{\text{op}} \times [\mathcal{B}^{\text{op}}, \mathbf{Cat}]_{\text{ps}} \longrightarrow \mathbf{Cat}$$

is defined by:

$$\begin{aligned} P(b, F) &= \mathbf{StrongTrans}(\mathcal{Y}_0(b), F), \\ P(f : b' \rightarrow b, \alpha : F \rightarrow G)(\eta) &= \alpha \circ \eta \circ \mathcal{Y}_0(f). \end{aligned}$$

The *evaluation pseudofunctor*

$$E : \mathcal{B}^{\text{op}} \times [\mathcal{B}^{\text{op}}, \mathbf{Cat}]_{\text{ps}} \longrightarrow \mathbf{Cat}$$

is defined by:

$$\begin{aligned} E(b, F) &= F(b), \\ E(f : b' \rightarrow b, \alpha : F \rightarrow G)(s) &= G(f)(\alpha_b(s)). \end{aligned}$$

4.2 The Equivalence

The main result is:

Theorem 4.3 (Bicategorical Yoneda Lemma). *There is a biequivalence*

$$P \simeq E$$

in the bicategory of pseudofunctors $\mathcal{B}^{\text{op}} \times [\mathcal{B}^{\text{op}}, \mathbf{Cat}]_{\text{ps}} \rightarrow \mathbf{Cat}$, i.e. strong transformations $\Phi : P \rightarrow E$ and $\Psi : E \rightarrow P$ together with invertible modifications $\Phi \circ \Psi \cong \text{Id}_E$ and $\Psi \circ \Phi \cong \text{Id}_P$.

The equivalence is witnessed by the following data.

The strong transformation $\Phi : P \rightarrow E$ has component

$$\Phi_{(b,F)} : \mathbf{StrongTrans}(\mathcal{Y}_0(b), F) \longrightarrow F(b), \quad \eta \longmapsto \eta_b(\text{id}_b).$$

That is, we evaluate the strong transformation η at the object b and then apply the resulting functor to the identity morphism $\text{id}_b \in \mathcal{B}(b, b)$.

The strong transformation $\Psi : E \rightarrow P$ has component

$$\Psi_{(b,F)} : F(b) \longrightarrow \mathbf{StrongTrans}(\mathcal{Y}_0(b), F), \quad s \longmapsto [(a, f) \mapsto F(f)(s)].$$

That is, the strong transformation assigned to $s \in F(b)$ sends an object $a \in \mathcal{B}^{\text{op}}$ and a morphism $f \in \mathcal{B}(a, b)$ to $F(f)(s)$.

Proposition 4.6. *The composites $\Phi \circ \Psi$ and $\Psi \circ \Phi$ are isomorphic to the respective identities via the modifications `yonedaInvHomId` and `yonedaHomInvId`. These are assembled from:*

- *The unit: for $\eta \in \mathbf{StrongTrans}(\mathcal{Y}_0(b), F)$, the component isomorphism at $f : a \rightarrow b$ is the composite $(\eta.\text{natural}(f))^{-1}|_{\text{id}_b} \circ \eta_a.\text{map}(\rho_f)$, where $\rho_f : f \circ \text{id}_b \cong f$ is the right unitor.*
- *The counit: for $s \in F(b)$, the component is $F.\text{mapId}(b)|_s$, the identity coherence of F applied to s .*

5 Universe Considerations

A subtle issue arises when comparing P and E as pseudofunctors into $\mathbf{Cat}$: they naturally land in different universe levels.

Fix universe variables u, v, w so that $\mathcal{B}$ has objects in $\mathbf{Type} \, u$, hom-categories with morphisms in $\mathbf{Type} \, v$, and 2-morphisms in $\mathbf{Type} \, w$. Then:

- $E'(b, F) = F(b)$ lands in $\mathbf{Cat}^{w,v}$ (i.e. a category with hom-sets in $\mathbf{Type} \, w$ and objects in $\mathbf{Type} \, v$).
- $P(b, F) = \mathbf{StrongTrans}(\mathcal{Y}_0(b), F)$ lands in $\mathbf{Cat}^{\max(u,v,w), \max(u,v,w)}$.

These two universe levels do not match, so the naive statement $P \simeq E'$ is not type-correct.

To resolve this we define an auxiliary pseudofunctor

$$\mathbf{catPseudoULift} : \mathbf{Cat}^{v_1, u_1} \rightarrow \mathbf{Cat}^{\max(v_1, v_2), \max(u_1, u_2)}$$

which promotes every small category to a larger universe by applying $\mathbf{ULift}$ on objects and $\mathbf{ULiftHom}$ on hom-sets. The actual evaluation pseudofunctor used in the theorem is

$$\mathbf{yonedaEvaluation} = \mathbf{yonedaEvaluation}' ; \mathbf{catPseudoULift},$$

where $\mathbf{yonedaEvaluation}'$ is the raw evaluation functor landing in $\mathbf{Cat}^{w,v}$. Universe-lifting is lossless: the equivalence $\mathbf{catLiftEquiv}$ witnesses $C \simeq \mathbf{catLift}(C)$ for every category C .

6 Design Choices in the Lean Formalization

6.1 The BiEquiv Structure

Rather than working with adjoint equivalences (which carry triangle identity data), we introduce a lighter structure sufficient for stating the Yoneda result:

```

1 structure BiEquiv (x y : B) where
2   map : x → y
3   inv : y → x
4   homInvId : map >> inv ≅ 1 x
5   invHomId : inv >> map ≅ 1 y

```

This records an internal equivalence in a bicategory — a pair of 1-morphisms with invertible 2-morphisms witnessing the round-trips — without requiring the triangle identities of an adjoint equivalence. Every adjoint equivalence gives a $\mathbf{BiEquiv}$, but the converse requires additional work.

6.2 Opacity and Definitional Reductions

Several auxiliary lemmas use `erw` (extended rewrite) rather than `rw` because the relevant equalities hold only after definitional unfolding of universe-lift wrappers such as `ULiftHom.up` and `ULift.upFunctor`. Replacing these with dedicated simp lemmas that expose the required reductions is a natural cleanup.

7 Future Work

The formalization established here opens several directions for further development of higher category theory in Lean 4 and Mathlib.

Biadjunctions. The biequivalence of Theorem 4.3 is the core ingredient in showing that the Yoneda embedding $\mathcal{Y} : \mathcal{B} \rightarrow [\mathcal{B}^{\text{op}}, \mathbf{Cat}]_{\text{ps}}$ is a *biadjunction* — the appropriate bicategorical analogue of a fully faithful functor admitting a left adjoint. Formalizing the general theory of biadjunctions in Mathlib, including the bicategorical adjoint functor theorem, would be a natural continuation [Lack, 2010].

The Grothendieck construction for bicategories. The classical Grothendieck construction builds a fibration over a category $\mathcal{C}$ from a functor $\mathcal{C} \rightarrow \mathbf{Cat}$. For bicategories there is an analogous *bicategorical Grothendieck construction*, producing a bicategoration over $\mathcal{B}$ from a pseudofunctor $\mathcal{B} \rightarrow \mathbf{Cat}$ [Street, 1974]. The pseudofunctors `yonedaPairing` and `yonedaEvaluation` are precisely the inputs this construction takes, so a formalization of bicategorical fibrations would sit naturally on top of the present work.

Pseudomonads and pseudoalgebras. A *pseudomonad* on a bicategory $\mathcal{B}$ is a monoid object in the hom-bicategory of pseudoendofunctors, with composition and unit isomorphisms satisfying coherence conditions. Pseudomonads model computational effects at a bicategorical level and include important examples such as the free cocompletion monad on $\mathbf{Cat}$. The Yoneda lemma is the key tool for showing that the free cocompletion of $\mathcal{B}$ is the bicategory of presheaves $[\mathcal{B}^{\text{op}}, \mathbf{Cat}]_{\text{ps}}$.

Acknowledgements

References

- Jean Bénabou. Introduction to bicategories. In *Reports of the Midwest Category Seminar*, pages 1–77. Springer, Berlin, 1967.
- Niles Johnson and Donald Yau. *2-Dimensional Categories*. Oxford University Press, 2021. Also available at <https://arxiv.org/abs/2002.06055>.
- Stephen Lack. A 2-categories companion. *Towards Higher Categories*, 152:105–191, 2010. Also available at <https://arxiv.org/abs/math/0702535>.
- Tom Leinster. *Higher Operads, Higher Categories*. Cambridge University Press, 2004. Also available at <https://arxiv.org/abs/math/0305049>.
- Ross Street. Fibrations and Yoneda’s lemma in a 2-category. *Lecture Notes in Mathematics*, 420:104–133, 1974.
- The Mathlib Community. Mathlib4: Mathematics library for Lean 4, 2024. URL <https://github.com/leanprover-community/mathlib4>.